# Chapter 1

# Nonlinear solid mechanics with axisymmetry

## 1.1 Theory

This example considers the solution of the PVD equations introduced in a previous tutorial, but in cylindrical coordinates and under the assumption of axisymmetry. In this case, the undeformed and deformed position vectors, denoted by $\mathbf{r}(\xi^i, t)$ and $\mathbf{R}(\xi^i, t)$ respectively, may be written as:

$$\mathbf{r}(\xi^i, t) = \begin{bmatrix} \xi^0 \cos(\xi^2) \\ \xi^0 \sin(\xi^2) \\ \xi^1 \end{bmatrix}, \qquad \mathbf{R}(\xi^i, t) = \begin{bmatrix} R(\xi^0, \xi^1, t) \cos(\xi^2) \\ Z(\xi^0, \xi^1, t) \sin(\xi^2) \\ \xi^1 \end{bmatrix}.$$

The undeformed and deformed covariant metric tensors, $g_{ij}$ and $G_{ij}$ respectively, are therefore:

$$g_{ij} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & {\xi^0}^2 \end{bmatrix}, \qquad G_{ij} = \begin{bmatrix} (\frac{\partial R}{\partial \xi^0})^2 + (\frac{\partial Z}{\partial \xi^0})^2 & \frac{\partial R}{\partial \xi^0}\frac{\partial R}{\partial \xi^1} + \frac{\partial Z}{\partial \xi^0}\frac{\partial Z}{\partial \xi^1} & 0 \\ \frac{\partial R}{\partial \xi^0}\frac{\partial R}{\partial \xi^1} + \frac{\partial Z}{\partial \xi^0}\frac{\partial Z}{\partial \xi^1} & (\frac{\partial R}{\partial \xi^1})^2 + (\frac{\partial Z}{\partial \xi^1})^2 & 0 \\ 0 & 0 & R^2 \end{bmatrix}$$

Recall the non-dimensional PVD equation, presented here with an additional mass damping term, $\eta_M \partial_t \mathbf{R}$:

$$\int_v \left\{ \sigma^{ij}\, \delta\gamma_{ij} - \left[ \mathbf{f} - \Lambda^2 \left( \frac{\partial^2 \mathbf{R}}{\partial t^2} + \eta_M \frac{\partial \mathbf{R}}{\partial t} \right) \right] \cdot \delta\mathbf{R} \right\} dv - \oint_{A_{tract}} \mathbf{T} \cdot \delta\mathbf{R}\, dA = 0. \qquad (1)$$

Note that in the implementation of these equations $\eta_M = 0$ by default. The following property holds due to the definition of the Green-Lagrange strain tensor $\gamma_{ij}$ and the symmetry of the second Piola stress tensor $\sigma^{ij}$:

$$\sigma^{ij}\delta\gamma_{ij} = \sigma^{ij}\delta\left[\frac{1}{2}(G_{ij} - g_{ij})\right] = \sigma^{ij}\frac{\partial \mathbf{R}}{\partial \xi^i} \cdot \delta\frac{\partial \mathbf{R}}{\partial \xi^j}.$$

Hence, the PVD equation becomes:

$$\int_v \left\{ \sum_{i,j=1}^{2} \sigma^{ij} \left[ \frac{\partial R}{\partial \xi^i}\frac{\partial(\delta R)}{\partial \xi^j} + \frac{\partial Z}{\partial \xi^i}\frac{\partial(\delta Z)}{\partial \xi^j} \right] + \sigma^{33} R \delta R - \left[ \mathbf{f} - \Lambda^2 \left( \frac{\partial^2 \mathbf{R}}{\partial t^2} + \eta_M \frac{\partial \mathbf{R}}{\partial t} \right) \right] \cdot \delta\mathbf{R} \right\} dv$$
$$- \oint_{A_{tract}} \mathbf{T} \cdot \delta\mathbf{R}\, dA = 0. \qquad (2)$$

The following example considers a steady state solution ( $\partial\mathbf{R}/\partial t = \partial^2\mathbf{R}/\partial t^2 = 0$) to these equations, though the solution to a time-dependent problem using the same underlying elements is discussed here.

## 1.2 Demo problem: Compression of a hyperelastic spherical shell

As a demonstration, we consider the compression of hyperelastic spherical shell which has an inner radius $a$ and thickness $h$ when stress-free, under an external pressure $p_{ext}$. The shell is assumed to obey the Neo-↩ Hookean consitutive law. For an incompressible material, the strain energy density (non-dimensionalised by the elastic modulus) is given by:

$$W = \frac{1}{4(1+\nu)}(I_1 - 3).$$

A generalization to the above, proposed by Attard ("Finite strain--isotropic hyperelasticity", International Journal of Solids and Structures 2003), for compressible materials is:

$$W = \frac{1}{2(1+\nu)}\left[\frac{1}{2}(I_1-3) - \frac{1}{2}\log(I_3) + \frac{\nu}{1-2\nu}\log\left(\sqrt{I_3}\right)^2\right].$$

In the above, $\nu$ is the Poisson ratio and $(I_1, I_2, I_3)$ are the strain invariants introduced previously. The incompressible and compressible cases shall now be considered seperately.

### 1.2.1 Incompressible case

When the material is incompressible, some analytical progress can be made by imposing spherical symmetry. Denoting $(\hat{r}, \theta, \phi)$ and $(\hat{R}, \Theta, \Phi)$ as the spherical coordinates in the undeformed and deformed configurations, respectively, the Cauchy principal stresses, denoted here as $S^{\hat{R}\hat{R}}$, $S^{\Theta\Theta}$ and $S^{\Phi\Phi}$ are given by:

$$S^{\hat{R}\hat{R}} = \frac{1}{2(1+\nu)}\lambda_{\hat{R}}^2 - p_s, \qquad S^{\Theta\Theta} = S^{\Phi\Phi} = \frac{1}{2(1+\nu)}\lambda_{\Theta}^2 - p_s,$$

where $\lambda_{\hat{R}} = d\hat{R}/d\hat{r}$ and $\lambda_\Theta = \lambda_\Phi = \hat{R}/\hat{r}$ are the principal stretches and $p_s$ is the solid pressure, the Lagrange multiplier term introduced in the solid theory tutorial to enforce incompressibility. The incompressibility condition is $\lambda_{\hat{R}}\lambda_{\Theta}^2 = 1$, from which the the dependence $\hat{R}(\hat{r})$ can be derived as:

$$\hat{R}^2 d\hat{R} = \hat{r}^2 d\hat{r} \longrightarrow \hat{R}_o^3 - \hat{R}_i^3 = (a+h)^3 - a_i^3,$$

where $\hat{R}_i$ and $\hat{R}_o$ denote the outer and inner radii of the shell, respectively, when under compression. A second equation for the two unknowns $\hat{R}_i$, $\hat{R}_o$ can be derived by first obtaining an expression for the solid pressure $p_s$ from the equilibrium equation:

$$\frac{dS^{\hat{R}\hat{R}}}{d\hat{R}} + \frac{2}{\hat{R}}(S^{\hat{R}\hat{R}} - S^{\Theta\Theta}) = 0 \longrightarrow p_s = \frac{1}{2(1+\nu)}\lambda_{\hat{R}}^2 + \frac{1}{1+\nu}\int\left(\frac{\hat{r}(\hat{R})^4}{\hat{R}^5} - \frac{\hat{R}}{\hat{r}(\hat{R})^2}\right)d\hat{R}$$

The integral can be evaluated by changing the integration variable to $\lambda_\Theta$ and using the incompressibility condition as follows:

$$\int\left(\frac{\hat{r}(\hat{R})^4}{\hat{R}^5} - \frac{\hat{R}}{\hat{r}(\hat{R})^2}\right)d\hat{R} = \int\left(\lambda_\Theta^{-4} - \lambda_\Theta^2\right)\left(\lambda_\Theta - \lambda_\Theta^4\right)^{-1}d\lambda_\Theta = -\left(\frac{1}{4\lambda_\Theta^4} + \frac{1}{\lambda_\Theta}\right) + const.$$

The radial principal Cauchy stress is therefore:

$$S^{\hat{R}\hat{R}} = \frac{1}{1+\nu}\left(\frac{1}{4\lambda_\Theta^4} + \frac{1}{\lambda_\Theta}\right) + const.$$

After applying the boundary conditions, $S^{\hat{R}\hat{R}} = 0$ on the inside face and $S^{\hat{R}\hat{R}} = -p_{ext}$ on the outside face, the second equation connecting $\hat{R}_i$ and $\hat{R}_o$ is found to be:

$$p_{ext} = \frac{1}{1+\nu}\left[\frac{1}{4}\left(\frac{a}{\hat{R}_i}\right)^4 + \frac{a}{\hat{R}_i} - \frac{1}{4}\left(\frac{a+h}{\hat{R}_o}\right)^4 - \frac{a+h}{\hat{R}_o}\right].$$

In the driver code discussed below, $\hat{R}_i$ and $\hat{R}_o$ are found via Newton iteration.

### 1.2.2 Compressible case

Although less analytical progress is possible in the compressible case, the problem can still be simplified down to a single ODE for $\hat{R}(\hat{r})$ using spherical symmetry. If the material obeys the compressible Neo-Hookean constitutive law, then the Cauchy principal stresses are:

$$S^{\hat{R}\hat{R}} = \frac{1}{J}\frac{1}{2(1+\nu)}\left[\lambda_{\hat{R}}^2 - 1 + \frac{2\nu}{1-2\nu}\log(J)\right], \qquad S^{\Theta\Theta} = \frac{1}{J}\frac{1}{2(1+\nu)}\left[\lambda_{\hat{\Theta}}^2 - 1 + \frac{2\nu}{1-2\nu}\log(J)\right],$$

where $J = \sqrt{I_3} = \lambda_{\hat{R}}\lambda_{\Theta}^2$. Substituting these expressions into the equilibrium equation gives:

$$\frac{d}{d\hat{R}}\left\{\frac{1}{J}\left[\lambda_{\hat{R}}^2 - 1 + \frac{2\nu}{1-2\nu}\log J\right]\right\} + \frac{2}{\hat{R}}\frac{1}{J}(\lambda_{\hat{R}}^2 - \lambda_{\Theta}^2) = 0.$$

In the undeformed frame, this equation reads:

$$\frac{d}{d\hat{r}}\left\{\frac{1}{J}\left[\lambda_{\hat{R}}^2 - 1 + \frac{2\nu}{1-2\nu}\log(J)\right]\right\} + \frac{2}{\hat{R}}\left[\left(\frac{\lambda_{\hat{R}}}{\lambda_{\Theta}}\right)^2 - 1\right] = 0 \qquad (3)$$

In this tutorial, a benchmark test is performed by solving equation (3) is solved seperately using 1D finite elements.

## 1.3 Results

The figure below shows plots of the deformed position $\hat{R}(\hat{r})$ against the undeformed position $\hat{r}$ for a range of Poisson ratio and external pressure values. The solid blue lines were obtained using the 2D axisymmetric elements, while the dashed blue lines were obtained under the assumption of spherical symmetry, following the approach outlined in sections Incompressible case (for $\nu = 0.5$) and Compressible case (for $\nu < 0.5$). Excellent agreement between the two approaches is found across all parameter sets considered.

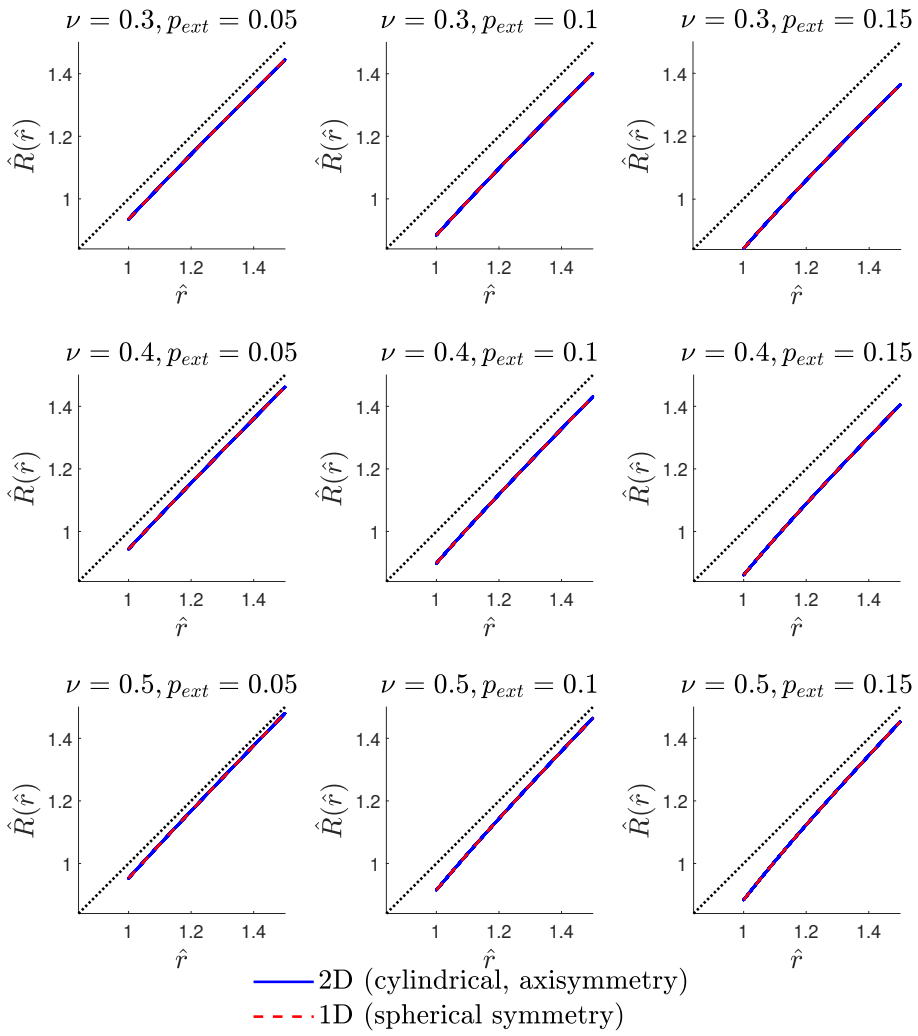



**Figure 1.1 Results of the benchmark test for various Poisson ratio and external pressure values.**

## 1.4 Global parameters and functions

As is standard, the namespaces `GlobalPhysicalVariables` and `GlobalSimSettings` are used to define the physical parameters and simulation parameters, respectively, for the problem.

```
//=======start_namespace==========================================
/// Global physical variables
//================================================================
namespace GlobalPhysicalVariables
{
  // Poisson ratio - only used in the compressible case
  // (nu = 0.5 for incompressible material)
  double nu = 0.4;

  // The external pressure
  double ext_pressure = 0.05;

  // Traction function for appyling a pressure on the outer surface
  void external_pressure_fct(const double& time,
                             const Vector<double>& xi,
                             const Vector<double>& x,
                             const Vector<double>& n,
                             Vector<double>& traction)
  {
    traction[0] = -1.0 * ext_pressure * n[0];
    traction[1] = -1.0 * ext_pressure * n[1];
  }
} // namespace GlobalPhysicalVariables

//=======start_namespace==========================================
```

```
/// Global simulation settings
//======================================================================
namespace GlobalSimSettings
{
  // Folder for output
  string result_folder = "RESLT";

  /// Flag for using a pressure formulation
  bool use_pressure_formulation = false;

  /// If using a pressure formulation, is the solid incompressible?
  bool incompressible = false;

  // Number of elements in radial direction
  unsigned n_radial = 10;

  // Number of elements in azimuthal direction
  unsigned n_azimuthal = 10;

  // Inner radius
  double a = 1.0;

  // Thickness
  double h = 0.5;
} // namespace GlobalSimSettings
```

## 1.5 The mesh

The mesh for this problem can be constructed simply by upgrading the existing `TwoDAnnularMesh` to a solid mesh.

```
//=====================start_mesh=================================
/// Upgrade the mesh to a solidmesh.
/// Quarter-annulus in quadrant r>0, z>0
/// ntheta - no. elements in theta direction
/// nr - no. elements in the radial direction
/// a - Inner radius of annulus
/// h - thickness of annulus
//======================================================================
template<class ELEMENT>
class ElasticTwoDAnnularMesh : public virtual TwoDAnnularMesh<ELEMENT>,
                               public virtual SolidMesh
{
public:
  /// Constructor: Build mesh and copy Eulerian coords to Lagrangian
  /// ones so that the initial configuration is the stress-free one.
  ElasticTwoDAnnularMesh<ELEMENT>(
    const unsigned& ntheta,
    const unsigned& nr,
    const double& a,
    const double& h,
    TimeStepper* time_stepper_pt = &Mesh::Default_TimeStepper)
    : RectangularQuadMesh<ELEMENT>(
        ntheta, nr, 1.0, 1.0, false, time_stepper_pt),
      TwoDAnnularMesh<ELEMENT>(false,
                               0.25,
                               ntheta,
                               nr,
                               a,
                               h,
                               MathematicalConstants::Pi,
                               time_stepper_pt)
  {
    // Make the current configuration the undeformed one by
    // setting the nodal Lagrangian coordinates to their current
    // Eulerian ones
    set_lagrangian_nodal_coordinates();
  }
}; // End mesh
```

## 1.6 The problem class

The driver contains two problem classes. The first is the main problem using the 2D axisymmetric elements, while the second is an auxiliary problem to solve equation (3) in the compressible case or find the unknowns $\hat{R}_i$ and $\hat{R}_o$

in the incompressible case. Since the latter of these is relatively standard, only the former is documented here. The problem class declaration is shown below.

```
//==============start_problem=============================================
/// Axisymmetric compression of a finite thickness shell in cylindrical
/// polar coordiantes
//========================================================================
template<class ELEMENT>
class ShellProblem : public Problem
{
public:
  /// Constructor:
  ShellProblem();

  /// Destructor
  ~ShellProblem() {}

  /// Doc the solution
  void doc_solution();

private:
  /// Create the boundary elements
  void create_boundary_elements();

  /// Delete boundary elements and wipe the associated mesh
  void delete_boundary_elements()
  {
    // How many surface elements are in the surface mesh
    unsigned n_element = Surface_mesh_pt->nelement();

    // Loop over the surface elements
    for (unsigned e = 0; e < n_element; e++)
    {
      // Kill surface element
      delete Surface_mesh_pt->element_pt(e);
    }

    // Wipe the mesh
    Surface_mesh_pt->flush_element_and_node_storage();
  } // end of delete_boundary_elements

  /// Complete the problem setup (attach function/physical parameter pointers)
  void complete_problem_setup();

  /// Bulk mesh
  ElasticTwoDAnnularMesh<ELEMENT>* Bulk_mesh_pt;

  /// Pointer to the "surface" mesh
  Mesh* Surface_mesh_pt;

  /// Pointer to constitutive law
  ConstitutiveLaw* Constitutive_law_pt;

  // DocInfo object
  DocInfo Doc_info;
};
```

## 1.7 The problem constructor

The constructor begins with assigning the output directory and creating the bulk and surface meshes. The function `create_boundary_elements()` is then called to create traction elements on the outer boundary of the shell.

```
//===============start_constructor========================================
/// Constructor for unstructured solid problem
//========================================================================
template<class ELEMENT>
ShellProblem<ELEMENT>::ShellProblem()
{
  // Set output directory
  Doc_info.set_directory(GlobalSimSettings::result_folder);

  // Initialise counter for solutions
  Doc_info.number() = 0;

  // Create the mesh: Quarter-annulus in the quadrant r>0, z>0.
  Bulk_mesh_pt =
    new ElasticTwoDAnnularMesh<ELEMENT>(GlobalSimSettings::n_azimuthal,
                                        GlobalSimSettings::n_radial,
                                        GlobalSimSettings::a,
                                        GlobalSimSettings::h);

  // Create the "surface mesh" that will contain only the interface
```

```
  // elements. The constructor just creates the mesh without giving
  // it any elements, nodes, etc.
  Surface_mesh_pt = new Mesh;

  // Create the boundary elements
  create_boundary_elements();

  // Add the two submeshes
  add_sub_mesh(Bulk_mesh_pt);
  add_sub_mesh(Surface_mesh_pt);

  // Combine all sub-meshes into a single mesh
  build_global_mesh();
```

Following this, a constitutive law is selected. Depending on the flag `GlobalSimSettings::is_incompressible`, this will either be the incompressible or compressible version of the NeoHookean constiutive law. The function `complete_problem_setup()` (discussed below) is then called and equation numbers are assigned.

```
  // Define a constitutive law from a strain energy density function.
  // Version depends on the incompressibility flag
  StrainEnergyFunction* strain_energy_function_pt;
  if (GlobalSimSettings::incompressible)
  {
    strain_energy_function_pt = new IncompressibleNeoHookean();
  }
  else
  {
    strain_energy_function_pt = new NeoHookean(&GlobalPhysicalVariables::nu);
  }
  Constitutive_law_pt =
    new IsotropicStrainEnergyFunctionConstitutiveLaw(strain_energy_function_pt);

  // Complete the problem set up
  complete_problem_setup();

  // Assign eqn numbers
  oomph_info << "Number of equations: " << assign_eqn_numbers() << std::endl;

} // end constructor
```

## 1.8 Completing the problem setup

To complete the problem setup, the pointer to the constitutive law is passed to each of the bulk elements. An attempt is made to cast to the version of the bulk element that contains pressure degrees of freedom, the `QAxisymmetricCylindricalPVDWithPressureElement`. If this cast is successful, then the run is using the solid pressure formulation and, depending on the value of `GlobalSimSettings::is_incompressible`, incompressibility may be enforced using the element's member function `set_incompressible()`. The pointer to the external pressure function is then passed to each of the interface elements for application of the traction boundary condition on the outer surface. Finally, The Dirichlet boundary conditions for this problem are set: pinned radial position at $r = 0$ and pinned axial position at $z = 0$.

```
//========start_of_complete_problem_setup================================
/// Assign function/parameter pointers, set boundary conditions etc.
//=======================================================================
template<class ELEMENT>
void ShellProblem<ELEMENT>::complete_problem_setup()
{
  // Determine number of bulk elements in mesh
  const unsigned n_element_bulk = Bulk_mesh_pt->nelement();

  // Loop over the bulk elements
  for (unsigned e = 0; e < n_element_bulk; e++)
  {
    // Upcast from GeneralisedElement to the present element
    ELEMENT* el_pt = dynamic_cast<ELEMENT*>(Bulk_mesh_pt->element_pt(e));

    // Set the constitutive law
    el_pt->constitutive_law_pt() = Constitutive_law_pt;

    // Cast to element with pressure dofs
    QAxisymmetricCylindricalPVDWithPressureElement* el_with_pres_pt =
      dynamic_cast<QAxisymmetricCylindricalPVDWithPressureElement*>(el_pt);

    // Can set compressible/incompressible condition here
    if (el_with_pres_pt != 0)
    {
```

```
      if (GlobalSimSettings::incompressible)
      {
        el_with_pres_pt->set_incompressible();
      }
      else
      {
        el_with_pres_pt->set_compressible();
      }
    }
  } // End of loop over bulk elements

  // Determine number of 1D interface elements in mesh
  const unsigned n_interface_element = Surface_mesh_pt->nelement();

  // Loop over the interface elements
  for (unsigned e = 0; e < n_interface_element; e++)
  {
    // Upcast from GeneralisedElement to the present element
    AxisymmetricCylindricalSolidTractionElement<ELEMENT>* el_pt =
      dynamic_cast<AxisymmetricCylindricalSolidTractionElement<ELEMENT>*>(
        Surface_mesh_pt->element_pt(e));

    // Set the traction function for applying pressure
    el_pt->traction_fct_pt() = &GlobalPhysicalVariables::external_pressure_fct;

  } // End of loop over interface elements

  // Set the Dirichlet boundary conditions for this problem
  // Find number of nodes on boundary 3
  unsigned n_boundary_node = Bulk_mesh_pt->nboundary_node(3);
  // Loop over nodes on fixed boundary
  for (unsigned n = 0; n < n_boundary_node; n++)
  {
    // Pin radial position on symmetry axis only
    Bulk_mesh_pt->boundary_node_pt(3, n)->pin_position(0);
  }
  n_boundary_node = Bulk_mesh_pt->nboundary_node(1);
  // Loop over nodes on fixed boundary
  for (unsigned n = 0; n < n_boundary_node; n++)
  {
    // Pin radial position on symmetry axis only
    Bulk_mesh_pt->boundary_node_pt(1, n)->pin_position(1);
  }
} // End complete_problem_setup
```

## 1.9 The driver code

The driver code begins by specifying command line arguments for whether the solid pressure formulation is to be used, whether incompressibility should be enforced, the value of the material's Poisson ratio and the external pressure applied on the outer face of the shell.

```
//==========start_main====================================================
/// Demonstrate how to solve an unstructured solid problem
//========================================================================
int main(int argc, char* argv[])
{
  // Store command line arguments
  CommandLineArgs::setup(argc, argv);

  string str_temp_solid_pressure;
  CommandLineArgs::specify_command_line_flag(
    "--use_solid_pressure", &str_temp_solid_pressure, "Use solid pressure");

  string str_temp_incompressible;
  CommandLineArgs::specify_command_line_flag(
    "--incompressible", &str_temp_incompressible, "Incompressible");

  CommandLineArgs::specify_command_line_flag(
    "--poisson", &GlobalPhysicalVariables::nu, "Poisson");

  CommandLineArgs::specify_command_line_flag(
    "--pressure", &GlobalPhysicalVariables::ext_pressure, "External pressure");

  CommandLineArgs::parse_and_assign();
  CommandLineArgs::doc_specified_flags();

  istringstream(str_temp_solid_pressure) >> std::boolalpha >>
    GlobalSimSettings::use_pressure_formulation;
  istringstream(str_temp_incompressible) >> std::boolalpha >>
    GlobalSimSettings::incompressible;
```

Following this, a number of consistency checks are made. For example, if incompressibility has been specified then a check is made that the solid pressure formulation has also been selected. If this is not the case, a warning is issued and the flag `GlobalSimSettings::use_pressure_formulation` is set to `true`.

```
  // If the Poisson ratio has been set to 0.5, but the
  // use_pressure_formulation and/or incompressible flags are false, set them to
  // true
  if (std::fabs(GlobalPhysicalVariables::nu - 0.5) < 1.0e-10 &&
      !(GlobalSimSettings::incompressible &&
        GlobalSimSettings::use_pressure_formulation))
  {
    std::string warning_message =
      "Inconsistency: Poisson ratio has been set to 0.5 but the  \n"
      "use_pressure_formulation and/or incompressible flags are false. \n"
      "Setting both of these to true to proceed.";
    OomphLibWarning(
      warning_message, OOMPH_CURRENT_FUNCTION, OOMPH_EXCEPTION_LOCATION);
    GlobalSimSettings::use_pressure_formulation = true;
    GlobalSimSettings::incompressible = true;
  }

  // If the incompressibility flag has been set, we must use the solid
  // pressure formulation. The Poisson ratio will not be used in this case.
  if (GlobalSimSettings::incompressible &&
      !GlobalSimSettings::use_pressure_formulation)
  {
    std::string warning_message =
      "Inconsistency: The incompressibility flag is true but we are not using "
      "\n"
      "the solid pressure formulation. Setting use_solid_pressure to true to "
      "proceed. \n";
    OomphLibWarning(
      warning_message, OOMPH_CURRENT_FUNCTION, OOMPH_EXCEPTION_LOCATION);
    GlobalSimSettings::use_pressure_formulation = true;
  }

  // Set the Poisson ratio to 0.5 if incompressibility has been specified. This
  // is solely for the purpose of output
  if (GlobalSimSettings::incompressible)
  {
    GlobalPhysicalVariables::nu = 0.5;
  }

  // End consistency checks
```

Finally, both the main and auxiliary problems are created and the Newton solve followed by output is done in each case.

```
  // Create the problem
  if (GlobalSimSettings::use_pressure_formulation)
  {
    // Build the problem with an additional degree of freedom for pressure
    ShellProblem<QAxisymmetricCylindricalPVDWithPressureElement> problem;

    problem.newton_solve();
    problem.doc_solution();
  }
  else
  {
    // Standard build
    ShellProblem<QAxisymmetricCylindricalPVDElement<3>> problem;

    problem.newton_solve();
    problem.doc_solution();
  }

  // Build and solve the auxiliary problem
  AuxiliaryShellProblem aux_problem;
  aux_problem.solve_auxiliary_problem();
  aux_problem.doc_solution();
} // end main
```

## 1.10 Source files for this tutorial

- The source files for this tutorial are located in the directory:

  `demo_drivers/axisym_solid/shell/`

- The driver code is:

demo_drivers/axisym_solid/shell/shell.cc

- A refineable version of the driver code can be found at:

demo_drivers/axisym_solid/shell/shell_adaptive.cc

## 1.11 PDF file

A pdf version of this document is available. \